Assignment 3: London Air Pollution

Group: Mohamed Salem, Zubair Abdul Matin, Mohammed Aziz, Salaah Shameem

Group Name: The 4 مَحَمَّدُ

Contributions

Teammate	Contributions
Zubair Abdul Matin	Zubair made the main window where everything is displayed. The main window includes the buttons to the different panels, along with filtering buttons. The main window also connects all the classes together and makes sure that the functionality works as intended. Zubair also implemented the API for the challenge tasks. He used openweathermap as the API. This API runs based on a latitude and longitude system. The API classes get the pollution data (NO2, PM2.5 and PM10) and the weather data (temperature, humidity and wind speed) from the API. The classes then parse the data into a suitable format. The function for returning this data could then be called where necessary, such as the InteractiveMap class that Mohammed Aziz implemented to get the pollution and weather data at those coordinates and display it on a tooltip.
Mohamed Salem	Mohamed Salem contributed mainly to making the MapController class. This class was responsible for the controls behind the map. This included loading the map, changing the pollution types, changing the year, displaying the pollution values as legends, making a tooltip for the pollution data so that exact values are shown, and more. He also made the PollutantThreshold Enum, which holds the high, low and average values of each pollutant. This information was then used to colour squares on the map as the legend. He also made the functionality for the filter buttons, such as changing between the maps and selecting specific legends of the data to be shown.
Salaah Shameem	Salaah implemented the stats section of the assignment. This included having a panel for the statistics. This statistics panel allows you to display the average pollution for every location on the map. It also shows the point on the map with the highest amount of a specific pollution. The statistics panel also has a button that can show the averages of the different pollutions, plotted on a graph, so that they can be compared with each other(this step takes a little while to load initially; please be patient). Salaah also implemented the Calculations class, which controls all the calculations with regard to the data within the program. He also implemented the Graph class, which controls how the line graphs are displayed on the screen.
Mohammed Aziz	Mohammed Aziz worked on many different parts of the program. This included doing all the styling on the program. Mohammed also implemented the InteractiveMap using the mapjfx library. This interactive map shows a map of the whole world and can be zoomed in and out of. Mohammed also worked on the welcome panel. The welcome panel is the first panel that shows on the window when the program is run. This panel details how the program works and how they can use it.

GUI and Functionality

The GUI consists mainly of four things. The main window controls all the GUI; it is where all the panels are accessed from through buttons. It is structured as a BorderPane, and in the top section, it has three buttons and the logo for our application. Those three buttons lead to either of the three panels: the welcome panel, the data map panel and the statistics panel.

The welcome panel only has the central part of the BorderPane, which contains a title along with information and pictures about the different functionalities of the software.

The data map panel has a left side and a centre side of the BorderPane. On the left side, it contains the filters and controls section. This section allows the person to choose different things for the map. It allows for the year, pollutant type and pollutant level (all, high, medium or low) to be chosen. The filters section also has two buttons that allow for switching to the live interactive map. One of those buttons is for the pollution live interactive map, and the other is for the weather live interactive map. As soon as the data map panel is opened, it displays the 2018 NO2 data onto the static London PNG. It also draws colours onto the map based on whether it has a low/medium/high level of pollution. There is also a tooltip that shows up whenever a place is hovered over.

The pollution stats panel is the last in the window, and it contains three sections. One of these sections allows the user to choose a starting and ending year for a specific pollution type, and it will display the entire map, but with the values/legend on the map corresponding to the averages of each of those points over the entire years. The next section allows the user to choose a specific pollutant and a specific year, and it displays the entire map with the highest pollution area highlighted in red. The last section allows the user to choose any of the pollution types or all three of them, and then it draws a graph of the selected pollution type with time on the x-axis and the average reading throughout London on the y-axis.

Testing

The class that we chose to do our unit testing on is the Calculation class. This class contains all the functionality for the datasets, such as finding all the relevant data within it for London, finding the maximum value in it, finding the average of all the data and more. Hence, it was the most suitable class to be tested. The way that we did the testing for this class was that we tested some of the methods and not all of them. This is because some methods in this class already call other methods within this class. So, by testing the method that calls the other, we would be testing the one which is called as well, unintentionally. So, there wouldn't be a need to redo the testing for that method

Types of testing performed

1. Core Mathematical Logic (findAverageFromData)

- Standard: Verified correct average calculation for multiple values.
- Single Element: Confirmed that the division logic handled one element/datapoint

correctly.
- Empty List: Verified that it returns Double.NaN when empty to prevent division by zero errors.

2. Spatial Filtering Logic (findRelevantData)

- Inside: Confirmed retention of points within the coordinate boundaries.
- Boundary: Validated inclusive logic (>=, <=) for points exactly on the boundary lines.
- Outside: Verified that points one unit outside the boundary are not included in the data.

3. Search Algorithms (maxLevel & locationOfMax)

- Standard Search: Confirmed correct identification of the highest value and its associated object.
- Empty State: Verified safe defaults (0.0 or null) when processing empty lists.

4. Integration Testing (findAverageFromSpecificYear)

- Pipeline: Validated the use of real parameters ("PM10", "2018").
- Data Validation: Confirmed CSV loading and filtering results match manually verified spreadsheet totals.

5. Coordinate Retrieval (findSpecificVal)

- Exact Match: Verified that the method successfully identifies and returns the specific value associated with precisely matching coordinates in the list.
- Not Found: Confirmed that a safe default of 0.0 is returned when the requested coordinates are not present within the dataset.

6. Data Accumulation Logic (addSum)

- Initialisation: Verified that newly encountered grid codes correctly trigger the creation of a map entry, initialising the sum array and setting the count to 1.0.
- Accumulation: Confirmed that existing grid codes are properly updated by aggregating new values into the running total and incrementing the frequency count.

7. Aggregation and Safety Checks (mergeMaps)

- Valid Merge: Validated the pipeline for calculating final averages (sum/count) and the generation of a new map linking DataPoints to their calculated results.
- Zero Count Edge Case: Ensured division-by-zero protections are in place to skip entries where the recorded count is 0.0, preventing potential system crashes.

8. Data Transformation (returnNewDataPoints)

- Object Mapping: Confirmed the transformation of a Map into a List of new DataPoint objects, ensuring coordinates remain unchanged while the new average is applied.

9. Multi-Year Integration Testing (findAverageFromAllYears)

- Batch Pipeline: Validated the full iteration across the required range of years (2018-2023) for specified pollutant types.
- Data Integrity: Confirmed the final output contains exactly six valid, non-zero entries, proving successful multi-file CSV loading and processing.

Possible Errors

We have verified that our code works as intended and that all the functionality is correct. However, certain things are out of our control, and we can't change them.

MacOS Library Compatibility: Due to OS-level compatibility issues with the underlying JavaFX WebView engine on the macOS environment, the interactive map components (using mapjfx/leaflet) fail to render on Mac devices. This is a documented limitation of the external libraries rather than a flaw in the application's logic, and it functions as intended on Windows environments.

Data Extrapolation Boundaries: The provided dataset consists of discrete geographical coordinates rather than continuous data. To generate the colored pollution map and legend, we implemented an extrapolation algorithm to represent the data across the static London map. Hence, some regions (specifically near the top border) are not covered by the colour overlays, which accurately reflect the spatial limits of the raw data provided.

BlueJ Dependencies: The project uses seven external dependencies to support the challenge tasks, all of which are included in the +libs folder. While the submitted .jar file contains everything needed to run, they **MUST DELETE** the "Module-info" file in the root of the reopened jar; the program wouldn't run unless that file is deleted.

API Rate Limiting: The real-time weather and pollution features rely on the OpenWeather API. This free-tier API has a strict daily quota for data calls. Should the application exceed this limit during extensive testing, the program will give the information as "N/A".